

חיפוש מיוחד

יוריסטיקה

יוריסטיקה זה משהו שיכולן אותנו ויגיד לנו בכל פעם איזה מצב לנסות בפעם הבאה. בקורס נדבר על יוריסטיקות שאומרות לנו "כמה אני קרוב למטרה?" היוריסטיקה היא הערכה - אין הבטחה שהיא תמיד הכי מדויקת או נכונה, אבל היא אמורה להיות:

1. מאוד מהירה

2. שימושית

למה אנחנו צריכים יוריסטיקות?

- מרחב החיפוש שלנו "מתפוצץ" - נהיה הרבה יותר מדי גדול.
- הרבה פעמים פתרון מדויק הוא NP-hard, או אפילו PSPACE-hard.
- יוריסטיקה נותנת כיוון - האם שווה להתקדם בכיוון מסוים? זה מאפשר לנו לפתח פחות קודקודי חיפוש.

- הגדרות: יוריסטיקה היא פונקציה $h : S \rightarrow \mathbb{N}_0$ (כאשר $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$)
- היוריסטיקה האופטימלית היא h^* שנותנת את העלות האמיתית של כל קודקוד.
- יוריסטיקה היא goal-aware אם $\forall g \in G, h(g) = 0$. כלומר אם הגענו ליעד, היא תגיד שהגענו ליעד.
- ההפך לא בהכרח נכון - היא יכולה לדווח 0 גם על קודקודים שאינם יעד.
- יוריסטיקה היא safe אם $h(s) = \infty \implies h^*(s) = \infty$. כלומר אם היא אומרת שאין דרך להגיע - בוודאות אין דרך להגיע.
- זה מאפשר לנו לא לפתח את הקדקדים האלה.
- יוריסטיקה היא admissible אם $\forall s \in S, h(s) \leq h^*(s)$. כלומר היא לעולם לא תעריך את העלות ביותר מאשר העלות האמיתית.
- יוריסטיקה היא consistent אם $\forall (u,v) \in E, h(u) \leq c(u,v) + h(v)$.
- למשל - אם מערכת ניווט אומרת שנשארו שתי דקות, ונסעתי דקה, אז לא יכול להיות שהיא תראה עכשיו יותר מדקה.

הגדרה: בהינתן שתי יוריסטיקות h_1, h_2 , h_1 שולטת (dominates) ב- h_2 אם:

$$\forall s, h_1(s) \geq h_2(s)$$

וגם שתיהן אדמיסיביליות.

¹תזכורת: $c(u,v)$ זה העלות להגיע מ- u ל- v .

Best First Search

Best First Search זה משפחה של אלגוריתמי חיפוש שבחרים כל פעם קודקוד לפתח את הקודקוד הכי טוב.

למשל: UCS

האלגוריתם מאפיינים ע"י פונקציה $f(n')$ שמגדירה איך האלגוריתם בוחר קודקוד חיפוש n' הכי טוב מבין כל הבנים של n . באמצעות פונציות שונות נוכל לקבל אלגוריתמים שונים מהמשפחה:

$$\bullet \text{ דייסטר } f(n) = g(n)$$

$$\bullet \text{ GBFS } f(n) = h(n)$$

$$\bullet \text{ } A^* \text{ } f(n) = g(n) + h(n)$$

Greedy Best-First Search

GBFS בוחר את הקודקוד הבא לפיתוח אך ורק לפי היוריסטיקה. כלומר - הוא מאמין לה, ולא מתחשב במסלול שהיה עד כה.

- תכונות:
- הוא לא מוצא מסלול אופטימלי.
 - אבל - הוא מוצא את המסלול מאוד מהר.
 - שובר שוויון באמצעות $g(n)$ או באמצעות FIFO.
 - אם היוריסטיקה מונוטונית, GBFS יתנהג דומה לDFS כי הקודקוד שיהיה שווה לפתח כל פעם יהיה בד"כ הבן של הקודקוד האחרון שפיתחנו.
 - לא שלם - אלא אם כן היוריסטיקה היא safe.

A^*

אלגוריתם שמתחשב גם במרחק שעברנו עד לקודקוד - וגם במרחק שנשאר ממנו. הוא בעצם שילוב של GBFS ו-UCS.

- תכונות:
- אם היוריסטיקה אדמיסיבילית, ואין קשתות שליליות, A^* יתן תשובה אופטימלית.
 - ככל שהיוריסטיקה יותר חזקה (יותר קרובה ל h^*), A^* יפתח פחות קודקודים.
 - אם h היא consistent², אזי $f(n)$ לאורך כל נתיב לא יורד - כל קודקוד צריך לפתח לכל היותר פעם אחת.

IDA^* (Iterative Deepening A^*)

A^* הוא אלגוריתם מאוד כבד מבחינת זיכרון. יכול להיות שנרצה לשלם בזמן ריצה (אבל לא בסיבוכיות אסימפטוטית!) בשביל לחסוך בזיכרון.

²אם יוריסטיקה היא consistent וגם goal-aware אז היא בהכרח admissible.

-
- הרעיון: נשתמש בDFS, אבל נפתח רק קודקודים בהם $f(n) \leq \text{threshold}$
 - אם לא מצאנו, נגדיל את threshold להיות ה- $f(n)$ של הקודקוד המינימלי מבין אלו שלא פיתחנו.
 - מתחילים מ- $\text{threshold} \leftarrow f(s_0) = g(s_0) + h(s_0) = h(s_0)$

חיפוש מקומי - Local Search Algorithms

Beam Search

- הרעיון: כמו Best-First Search, אבל שומרים בכל פעם רק מספר מצומצם של קודקודים בערימת הקודקודים לפיתוח (limited frontier)
- האלגוריתם לא שלם - הוא עלול לזרוק את כל הקודקודים שהיה אפשר להגיע מהם לפתרון.
- אפשר:
- להשתמש בו כמו GBFS $f(n) = h(n)$. זה נקרא Greedy Beam Search. זאת הגרסה הסטנדרטית.
 - להשתמש בו כמו A^* עם $f(n) = g(n) + h(n)$
 - Weighted Beam: $f(n) = g(n) + w \cdot h(n)$ עבור $w \geq 1$.